Control Plane Engineer Assignment: RADIUS Accounting Server

Problem Statement

As a Control Plane Engineer, you are tasked with implementing a RADIUS (Remote Authentication Dial-In User Service) server that can process accounting information and store this data for analysis. This exercise will test your understanding of network protocols, accounting mechanisms, and data persistence in distributed systems. **Note:** This server is intended to receive a copy/mirror of the live RADIUS traffic for monitoring and data collection purposes; it will not be performing the actual authentication itself.

Background

RADIUS is a networking protocol that provides centralized Authentication, Authorization, and Accounting (AAA) management for users who connect to and use network services. It is commonly used in enterprise networks, ISPs, and wireless networks to manage access to the network infrastructure. This assignment focuses specifically on the **Accounting** aspect.

Requirements

1. **RADIUS Server Implementation**
    - Develop a RADIUS server in Go that listens on the standard RADIUS accounting port (1813)
    - Implement support for processing Accounting-Request messages and generating appropriate Accounting-Response messages.
    - Configure the server with a shared secret key for RADIUS message verification
    - **Use the layeh.com/radius package** for RADIUS protocol implementation
2. **Accounting Data Processing**
    - Extract key accounting attributes from incoming RADIUS Accounting-Request packets.
    - Process different accounting types (e.g., Start, Stop, Interim-Update).
3. **Data Storage**
    - Store all RADIUS accounting records in Redis.
    - Each record should include key attributes from the RADIUS request (username, NAS-IP-Address, Acct-Status-Type, Acct-Session-Id, timestamp, etc.)
    - Generate a unique key for each accounting record using a suitable format, e.g., `radius:acct:{username}:{acct_session_id}:{timestamp}`
    - Store the data as JSON objects with relevant fields extracted from the accounting packet, such as:
        - `username` : The username from the RADIUS request (User-Name attribute)
        - `nas_ip_address` : The IP address of the Network Access Server (NAS-IP-Address attribute)

        - `nas_port` : The port number on the NAS (NAS-Port attribute)
        - `acct_status_type` : The type of accounting record (Start, Stop, etc.) (Acct-Status-Type attribute)
        - `acct_session_id` : Unique identifier for the session (Acct-Session-Id attribute)
        - `framed_ip_address` : IP address assigned to the user (Framed-IP-Address attribute)
        - `calling_station_id` : The caller's identifier (Calling-Station-Id attribute)
        - `called_station_id` : The called party's identifier (Called-Station-Id attribute)
        - `timestamp` : When the accounting request was received
        - `client_ip` : The IP address of the client making the request
        - `packet_type` : The type of RADIUS packet (e.g., "Accounting-Request", "Accounting-Response")
    - Set a time-to-live (TTL) for each record (e.g., 24 hours)
    - Implement proper error handling for Redis storage operations

4. **Testability**

    - Ensure the server can be tested with standard RADIUS testing tools like `radclient`
    - Implement appropriate logging for debugging and monitoring

5. **Containerization**

    - Dockerize the solution with appropriate Dockerfile configuration for the RADIUS server.
    - Create a `docker-compose.yml` file that includes the RADIUS server, the Redis service, **and Redis Subscriber service (see Requirement 6)**.
    - Ensure proper networking between containers.
    - Configure appropriate volume mappings for Redis persistence **and log file persistence for the subscriber service**.
    - The final `docker-compose` setup should orchestrate the following four containers:
        a. **`radius-controlplane`** : The main Go application that listens on port 1813, receives RADIUS Accounting-Request packets, stores data in Redis, and sends Accounting-Response packets.
        b. **`redis`** : The Redis database instance, configured to store accounting data and enable Keyspace Notifications (`notify-keyspace-events KEA`).
        c. **`redis-controlplane-logger`** : The second Go application that subscribes to Redis Keyspace Notifications (specifically for keys set by the `radius-controlplane` service) and logs the key and its value to a persistent log file.
        d. **`radclient-test`** : A container with `freeradius-utils` installed, used to send test `radclient` commands (Accounting-Start, Accounting-Stop) to the `radius-controlplane` service. This container can be used for manual testing by accessing its shell (`docker-compose exec radclient-test bash`).

6. **Redis Subscriber Service**

    - Develop a **separate Go application** that acts as a Redis subscriber.
    - This service should subscribe to Redis Keyspace Notifications, specifically listening for events related to the keys created by the RADIUS accounting server (e.g., `SET` events on keys matching the pattern `radius:acct:*`).
    - **Note:** Redis Keyspace Notifications might need to be explicitly enabled in the Redis configuration (e.g., `notify-keyspace-events KEA` or similar).

- Upon receiving a notification that a key has been set (indicating a new accounting record was stored), the subscriber service should:
  - Log a message to a specified file (e.g., `/var/log/radius_updates.log`).
  - The log message should include a timestamp and the name of the key that triggered the notification. Example format: `YYYY-MM-DD HH:MM:SS.ffffff - Received update for key: <key_name>`.
- Ensure the subscriber handles potential connection errors with Redis gracefully and attempts to reconnect.
- Dockerize this subscriber service.

Testing with radclient

The server should be tested using `radclient`, a command-line tool from the FreeRADIUS package. This tool can send arbitrary RADIUS packets, including Accounting-Request packets, to a specified server and displays the responses.

Installing radclient

`radclient` is typically included in the same package as `radtest`.

```
# On Ubuntu/Debian
sudo apt-get update && sudo apt-get install -y freeradius-utils

# On macOS with Homebrew
brew install freeradius-server
```

Basic Usage

`radclient` reads packet information from standard input or a file. To send an Accounting-Request, you need to specify the necessary attributes.

Create a file (e.g., `acct_start.txt`) with the following content:

```
User-Name = "testuser"
Acct-Status-Type = Start
Acct-Session-Id = "session12345"
NAS-IP-Address = 192.168.1.1
NAS-Port = 0
Calling-Station-Id = "123-456-7890"
Called-Station-Id = "987-654-3210"
Framed-IP-Address = 10.0.0.100
```

Then, send the packet using `radclient`:

```
radclient -x localhost:1813 acct testing123 < acct_start.txt
```

Example Test Cases

1. **Test Accounting Start:** Create `acct_start.txt` as shown above and run:

```
radclient -x localhost:1813 acct testing123 < acct_start.txt
```

Expected output shows an Accounting-Response.

2. **Test Accounting Stop:** Create `acct_stop.txt` with `Acct-Status-Type = Stop` and other relevant attributes (like Acct-Input-Octets, Acct-Output-Octets, Acct-Session-Time):

```
User-Name = "testuser"
Acct-Status-Type = Stop
Acct-Session-Id = "session12345"
NAS-IP-Address = 192.168.1.1
NAS-Port = 0
Calling-Station-Id = "123-456-7890"
Called-Station-Id = "987-654-3210"
Framed-IP-Address = 10.0.0.100
Acct-Input-Octets = 1000000
Acct-Output-Octets = 500000
Acct-Session-Time = 3600
```

Run:

```
radclient -x localhost:1813 acct testing123 < acct_stop.txt
```

Expected output shows an Accounting-Response.

After running these tests, you should verify:

1. The server correctly receives and processes Accounting-Request packets.
2. The server sends back appropriate Accounting-Response packets.
3. The accounting data is properly stored in Redis with the correct attributes.
4. The logs show appropriate information about the accounting requests received.

Expected Deliverables

1. Complete Go implementation of the RADIUS accounting server with Redis integration.
2. **Complete Go implementation of the Redis subscriber service.**
3. Dockerfile for the RADIUS server, **Dockerfile for the subscriber service,** and `docker-compose.yml` for the entire solution (including Redis).
4. Documentation explaining your approach, design decisions, and how to run the services.
5. Instructions for testing the server using `radclient`.
6. Instructions for building and running the containerized solution **(including verifying the subscriber's log output)**.

Evaluation Criteria

Your solution will be evaluated based on:

1. **Functionality**: Does the server correctly implement the RADIUS accounting protocol? **Does the subscriber correctly receive and log Redis notifications?**
2. **Code Quality**: Is the code well-structured, readable, and maintainable for **both Go applications**?
3. **Error Handling**: Do the applications handle errors gracefully (including network/Redis issues) and provide meaningful feedback?
4. **Performance**: Is the solution efficient and able to handle multiple concurrent requests/notifications?
5. **Data Integrity**: Is accounting data accurately extracted and stored? **Are notifications reliably processed?**
6. **Containerization**: Is the solution properly containerized, configured, and easy to deploy via `docker-compose` ?
7. **Timeliness**: Is the solution delivered within the committed timeframe?

Use of AI

If any artificial intelligence (AI) tools are utilized during the completion of this assignment, such usage must be fully disclosed and clearly documented, including:

1. **Type of AI Tool**: Specify the category and name of the AI tool employed
2. **Usage Methodology**: Describe how the AI tool was utilized throughout the development process
3. **AI-Generated Deliverables**: Identify which specific deliverables were created or significantly influenced with assistance from AI tools

Setup Instructions

To get started with this assignment:

1. Ask any questions you need to understand the assignment; you may ask questions any time during your work on the assignment.
2. Inform us when you will finish the assignment before you start working on it.
3. Implement the RADIUS accounting server according to the requirements
4. Create Docker and docker-compose configurations
5. Test your implementation both locally and in containers
6. Document your solution with clear instructions for running with docker-compose.
7. Document the use of AI in the assignment.

Resources

- RADIUS Accounting RFC 2866
- Go Redis Client Documentation
- Redis Keyspace Notifications
- RADIUS in Go (layeh.com/radius)
- Docker Documentation
- Docker Compose Documentation
- FreeRADIUS Documentation

Good luck with your implementation! Feel free to ask any questions if requirements are unclear.